

1. Create a database CollegeDB and a collection students. Insert student documents containing roll number, name, department, year, and marks. Perform CRUD operations on the documents.
2. Using aggregation, find the **average marks department-wise**, display the department and average marks, sort by average marks in descending order, and limit the result to 3 departments.

1. Create a database HospitalDB and collection patients. Insert patient details such as patient ID, name, age, gender, disease, and doctor. Perform insert, query, update, and delete operations.
2. Use aggregation to **match patients with a specific disease**, group them by doctor, count the patients, and sort the result based on patient count.

1. Create LibraryDB with a books collection.

Insert book details including ISBN, title, author, category, price, and availability. Perform CRUD operations.

2. Use \$match, \$project, \$sort, and \$limit to display the **five most expensive available books** with only title, author, and price.

1. Create BankDB and an accounts collection. Insert customer account details including account number, customer name, account type, balance, and branch. Update and delete selected account documents.

2. Use aggregation to calculate the **average account balance branch-wise**, sort the branches by average balance, and display the top 5 branches.

1. Create ShopDB and a products collection. Insert product ID, name, category, brand, price, and stock details. Perform CRUD operations.

2. Use an aggregation pipeline to **match products belonging to a particular category**, project required fields, sort by price, and display the first 10 products.

1. Create EmployeeDB with an employees collection. Insert employee ID, name, department, designation, salary, and experience. Perform insert, query, update, and delete operations.
2. Group employees by department and calculate their **average salary and total number of employees**. Sort departments by average salary.

1. Create MovieDB and a movies collection. Insert movie name, director, genre, year, rating, and actors. Perform CRUD operations.
2. Use \$match, \$sort, and \$limit to find the **top 5 highest-rated movies released after 2020**.

1. Create TravelDB and a tourists collection. Insert tourist ID, name, destination, age, country, and package cost. Perform CRUD

operations.

2. Calculate the **average package cost for each destination**, display destination and average cost, and sort the result in descending order.

1. Create HotelDB and a bookings collection. Insert booking ID, customer name, hotel, room type, number of nights, and amount. Perform CRUD operations.
2. Use aggregation to find the **total booking amount for each hotel**, sort hotels based on total revenue, and limit the output to 5 hotels.

1. Create EcommerceDB with an orders collection. Insert order ID, customer, product, category, quantity, price, and order date. Perform CRUD operations.
2. Use \$project to calculate order value (quantity $\times$ price), group orders by category, and find the **average order value for each category**.

1. Create SportsDB and a players collection. Insert player name, team, sport, age, matches,

runs/goals, and country. Perform CRUD operations.

2. Group players by team and calculate the **average performance score**. Sort teams based on their average performance and display the top 3 teams.

1. Create SchoolDB with a students collection containing student details and an array of enrolled courses. Insert, update, query, and delete student documents.

2. Use \$unwind to display each student's courses separately. Group by course and find the **number of students enrolled in each course**.